Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization

Agents That Reason in Activation Space Rather Than Text Evaluate 5–13× More Programs Per Hour on Mathematical and Scientific Discovery

By Qile Jiang, George Em Karniadakis
arXiv:2611.08483 • cs.AI / cs.NE • Brown University

LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13× more programs per hour than state-of-the-art frameworks and reaching equivalent solution quality in 3–8 minutes versus 60 minutes for competing systems.

AT A GLANCE

Problem: LLM-guided evolutionary search is accurate but too slow for large-scale discovery; each round requires full chain-of-thought for every candidate.

Why it matters: Scientific computing and mathematical discovery require evaluating hundreds of program variants per experiment; throughput is a hard bottleneck.

Method: 10 swarms of 45 latent agents share KV caches and reason for 40 latent steps without decoding to text; a lightweight decoding head maps top agent states to program tokens.

Result: 5–13× more programs per hour; solution quality reached in 3–8 min vs. 60 min for text-based frameworks.

Evidence: 21 problems spanning symbolic math and PDE solver kernel optimization; baselines: OpenEvolve, ShinkaEvolve, CodeEvolve.

THE BIG PICTURE

Program evolution with LLMs (OpenEvolve, FunSearch, ShinkaEvolve) has produced surprising discoveries in combinatorics and numerical computing by using LLMs as intelligent mutation operators. The approach works because LLMs encode prior knowledge about program structure, algorithm design, and mathematical transformations that random mutation cannot access.

The fundamental bottleneck is throughput. Generating a new program candidate with chain-of-thought reasoning costs one full LLM forward pass with a long context window. At 450 tokens per second on typical hardware, a 1,000-token chain-of-thought allows only 27 candidates per minute per GPU. SwarmEvolve attacks this bottleneck by eliminating the decoding step from most of the computation.

WHY EXISTING APPROACHES FALL SHORT

Text-based frameworks (OpenEvolve, CodeEvolve) generate each candidate as a full text completion—flexible but slow. **Parallel sampling with temperature** can explore multiple candidates simultaneously but each sample is independent and cannot condition on what other concurrent samples produce. **Smaller surrogate models** are faster but lack reasoning quality, leading to lower-quality proposals and slower convergence.

CORE MECHANISM: LATENT AGENTS WITH KV SHARING

Each agent is a frozen LLM conditioned on a shared prefix KV cache covering the problem description and global best solution. The agent then runs 40 autoregressive steps *without* applying the unembedding matrix: no token is produced. Instead, the final hidden state $h_i \in \mathbb{R}^d$ encodes the agent's proposal in activation space.

Agents in the same swarm share a common prefix KV cache, eliminating redundant computation. Different random seed vectors (appended to the shared prefix) induce diversity within the swarm. A lightweight decoding head maps the top-*r* agents' hidden states to program token sequences.

TECHNICAL FORMULATION

Let K = 10 swarms each containing N = 45 agents. The shared prefix for swarm k is:

```
K_k = KVCache([problem, x*_k])
```

where x^*_k is the globally best program found so far. Each agent produces:

```
h_{k,i} = f_θ(z_{k,i}, K_k) ∈ ℝ^d
```

The fitness-weighted swarm centroid is `ħ_k = ∑w_i h_{k,i} / ∑w_i` with weights `w_i = softmax(F_{k,i}/T)`. The top-r = 5 agents by cosine similarity to the centroid are decoded.

LEARNING OR INFERENCE PROCEDURE

The decoding head D_φ is a 2-layer MLP trained offline on (hidden-state, program) pairs extracted from text-based evolutionary runs. At inference time D_φ is frozen; SwarmEvolve requires no online training. The evolutionary loop per round: (1) Update all K prefix caches; (2) Run all 450 agents in parallel (40 latent steps); (3) Select top-r agents per swarm; (4) Decode to programs via D_φ; (5) Execute and score; (6) Update global bests.

WHAT THE GUARANTEE SAYS

No formal convergence guarantee is provided. The empirical claim is throughput superiority on 21 specific problems. Best-known solutions to 11 of the 21 problems (symbolic math subset) have formal certificates; SwarmEvolve recovers certified-correct solutions for 9 of these 11 within the 2 GPU-hour budget.

EXPERIMENTAL FINDINGS

- **Throughput:** 5–13× more programs per hour than all three baselines.
- **Time-to-solution:** Median 3–8 minutes vs. 60 minutes on the same hardware.
- **Quality:** Highest average score across 21 problems; wins 17 of 21 head-to-head.
- **Diversity:** 10-swarm config produces 1.9× more distinct program families per hour than a 450-agent single-swarm variant.

ABLATIONS AND INTERPRETATION

Latent steps: 40 is near-optimal; 20 steps reduces quality by 4.3 points; 80 shows no improvement. **Swarm count:** 10 swarms outperform 5 by 2.7 points and 1 swarm by 7.1 points. **KV sharing:** Removing the shared prefix cache reduces throughput 2.3× and quality 3.2 points. **Decoding head:** Replacing D_φ with a linear probe reduces program validity from 94% to 71%.

REFERENCE

Qile Jiang, George Em Karniadakis. **SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization.** arXiv:2611.08483. arxiv.org/abs/2611.08483